

OS LAB-8
DATE: 4/5/26

SUMANT SHRIDHAR
USN: 1BM24CS299
BATCH: 4K2

1) Dining Philosopher Problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t chopstick[N];
int ids[N]; // Philosopher IDs

void* philosopher(void* arg) {
    int id = *(int*)arg;

    while (1) {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        printf("Philosopher %d is waiting\n", id);

        // Pick up
        sem_wait(&chopstick[id]); //try to pick up left chopstick
        sem_wait(&chopstick[(id + 1) % N]); //try right

        printf("Philosopher %d is eating\n", id);
        sleep(1);

        // Put down
        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id + 1) % N]);

        printf("Philosopher %d finished eating\n", id);
    }

    return NULL;
}

int main() {
    pthread_t p[N];

    for (int i = 0; i < N; i++) {
        sem_init(&chopstick[i], 0, 1); //1 is chopstick available
        ids[i] = i; //assign ids
    }
}
```

```
for (int i = 0; i < N; i++) {  
    pthread_create(&p[i], NULL, philosopher, &ids[i]);  
}  
  
for (int i = 0; i < N; i++) {  
    pthread_join(p[i], NULL);  
}  
  
return 0;  
}
```

```
Philosopher 0 is thinking  
Philosopher 1 is thinking  
Philosopher 2 is thinking  
Philosopher 3 is thinking  
Philosopher 4 is thinking  
Philosopher 1 is waiting  
Philosopher 1 is eating  
Philosopher 2 is waiting  
Philosopher 3 is waiting  
Philosopher 3 is eating  
Philosopher 0 is waiting  
Philosopher 4 is waiting  
Philosopher 1 finished eating  
Philosopher 1 is thinking
```

2) Deadlock Detection Algorithm

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main() {  
    int n, m, i, j;
```

```
  
    int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX], available[MAX];  
    int finish[MAX] = {0}, safeSeq[MAX];  
    int count = 0;
```

```
  
    printf("Enter number of processes: ");  
    scanf("%d", &n);
```

```
  
    printf("Enter number of resources: ");  
    scanf("%d", &m);
```

```
  
    printf("Enter Allocation Matrix:\n");  
    for (i = 0; i < n; i++) {  
        for (j = 0; j < m; j++) {  
            scanf("%d", &alloc[i][j]);  
        }  
    }
```

```
  
    printf("Enter Maximum Matrix:\n");  
    for (i = 0; i < n; i++) {  
        for (j = 0; j < m; j++) {  
            scanf("%d", &max[i][j]);  
        }  
    }
```

```
  
    // Calculate Need Matrix  
    for (i = 0; i < n; i++) {  
        for (j = 0; j < m; j++) {  
            need[i][j] = max[i][j] - alloc[i][j];  
        }  
    }
```

```
  
    printf("Enter Available Resources:\n");  
    for (j = 0; j < m; j++) {  
        scanf("%d", &available[j]);  
    }
```

```
  
    int done, k;
```



```

do {
    done = 0;
    for (i = 0; i < n; i++) {
        if (finish[i] == 0) {
            int can_execute = 1;
            for (j = 0; j < m; j++) {
                if (need[i][j] > available[j]) {
                    can_execute = 0;
                    break;
                }
            }
            if (can_execute) {
                for (k = 0; k < m; k++) {
                    available[k] += alloc[i][k];
                }
                finish[i] = 1;
                safeSeq[count++] = i; // record process in safe sequence
                done = 1;
            }
        }
    }
} while (done);

// Check for deadlock
int deadlock = 0;
printf("\nProcesses in Deadlock:\n");
for (i = 0; i < n; i++) {
    if (finish[i] == 0) {
        printf("P%d ", i);
        deadlock = 1;
    }
}

if (!deadlock) {
    printf("No Deadlock\n");
    printf("Safe Sequence: ");
    for (i = 0; i < count; i++) {
        printf("P%d ", safeSeq[i]);
    }
    printf("\n");
}

return 0;
}

```

```
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0
1
0
2
0
0
3
0
2
2
1
1
0
0
2
Enter Maximum Matrix:
7
5
3
3
2
2
9
0
2
2
2
2
4
3
3
Enter Available Resources:
3
3
2

Processes in Deadlock:
No Deadlock
Safe Sequence: P1 P3 P4 P0 P2
```


Lab 8

classmate

Date _____
Page _____

4/5/26

Dining Philosopher Problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t chopstick[N];
int ids[N]; // philosophers

void* philosopher(void* arg) {
    int id = *(int*)arg;

    while (1) {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        printf("Philosopher %d is waiting\n", id);

        // pick up sticks
        sem_wait(&chopstick[id]); // left
        sem_wait(&chopstick[(id+1)%N]); // right
        printf("Philosopher %d is eating\n", id);
        sleep(1);

        // Put down sticks
        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id+1)%N]);

        printf("Philosopher %d finished eating\n", id);
    }

    return NULL;
}
```



```

int main() {
    pthread_t p[N];
    for (int i = 0; i < N; i++) {
        sem_init(&chopstick[i], 0, 1);
        ids[i] = i;
    }

    for (int i = 0; i < N; i++) {
        pthread_create(&p[i], NULL,
                      philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(p[i], NULL);
    }

    return 0;
}

```

Op:

Philosopher 0 is thinking
 Philosopher 1 is waiting
 Philosopher 2 is eating
 Philosopher 3 is thinking
 Philosopher 4 finished eating
 Philosopher 4 is waiting
 Philosopher 2 is eating

Deadlock Detection

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main()
```

```
{ int n, m, i, j;
```

```
int alloc[MAX][MAX], max need[MAX][MAX] available
```

```
int finish[MAX] = {0}; safeSeq[MAX];
```

```
int count = 0;
```

```
printf("Enter no. of processes: ");
```

```
scanf("%d", &n);
```

```
printf("Enter no. of resources: ");
```

```
scanf("%d", &m);
```

```
printf("Enter Allocation Matrix: \n");
```

```
for (i = 0; i < n; i++) {
```

```
for (j = 0; j < m; j++) {
```

```
scanf("%d", &alloc[i][j]);
```

```
}
```

```
}
```

```
printf("Enter Request Maximum Matrix: \n");
```

```
for (i = 0; i < n; i++) {
```

```
for (j = 0; j < m; j++) {
```

```
scanf("%d", &request[i][j]);
```

```
}
```

```
}
```

```
printf("Enter Available Resources: \n");
```

```
for (j = 0; j < m; j++) {
```

```
scanf("%d", &available[j]);
```

```
}
```

```
int done[k];
```

```
for (i = 0; i < n; i++) {
```

```
for (j = 0; j < m; j++) {
```

```
need[i][j] = max[i][j] - alloc[i][j];
```

```
}
```

```
}
```



```

do {
    done = 0;
    for (i = 0; i < n; i++) {
        if (finish[i] == 0) {
            int can_exec = 1;
            for (j = 0; j < m; j++) {
                if (need[i][j] > available[j]) {
                    can_exec = 0;
                    break;
                }
            }
            if (can_exec) {
                for (k = 0; k < m; k++) {
                    available[k] += alloc[i][k];
                }
                finish[i] = 1;
                done = 1;
                safe_count++;
            }
        }
    }
} while (done);

```

// Check for deadlock

```

int dlock = 0;
for (i = 0; i < n; i++) {
    if (finish[i] == 0) {
        printf("P%d", i);
        dlock = 1;
    }
}

```

if (!dlock) pf "No Deadlock"

~~OP:~~

Op:
Enter no
Enter no
Enter A
2 0
3 0
2 1
0 0
Enter
3 2
9 0
2
4
Enter

No De
Safe S

S


```

Q1: pf("Safe Sequence: ")
for li = 0; i < count; i++)
    pf("P %d", safeSeq[i]);
}
pf("\n");
return 0;
}

```

O/p:

Enter no of processes: 5

Enter no of resources: 3

Enter Allocation Matrix: 0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Matrix: 7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources: 3 3 2

~~Ans~~

No Deadlock

Safe Sequence: P1 P3 P4 P0 P2

S.P. 1
 11/5/20.